FOBO AI

Football Outcome Betting Optimiser

Pipeline Architecture & Design Rationale

This document provides a complete technical description of the FOBO AI pipeline: from raw web-scraped match data through deep learning, reinforcement learning, and gradient-boosted ensemble models, terminating in a Flask-served prediction API. Each architectural decision is justified in the context of football's statistical properties.

1. System Overview

FOBO AI is a multi-stage football match prediction system. It ingests historical results from across 13 European leagues, trains a hybrid deep learning + tree-based ensemble, and applies a reinforcement learning agent to decide when a bet is mathematically justified. The final output is served via a web dashboard.

Figure 1 — End-to-end pipeline. Each stage feeds the next.

Stage	Module	Purpose
① Scraping	scrape_flashscore.py	Parallel HTML scraping of match results, odds, xG
② Validation	check_data.py	CSV integrity, required columns, date parsing
③ Seq Optimisation	optimize_seq_length.py	Tune per-league LSTM look-back window via grid search
④ DL Training	train_dl.py	Train LeagueAwareModel with Dixon-Coles NLL loss
⑤ PPO RL	prediction_model.py	Fine-tune bet decision agent with Kelly reward
⑥ Hybrid Ensemble	train_hybrid.py	XGBoost + LightGBM on DL embeddings
⑦ Calibration	train_hybrid.py	Platt/Sigmoid scaling of XGBoost probabilities
⑧ Flask API	app.py	REST endpoints + interactive frontend

2. Data Layer

Data is scraped from FlashScore using a parallelised Selenium-based scraper. Each league produces a CSV file (*_RESULTS.csv) containing match date, home/away teams, full-time score, pre-match odds (1/X/2), and expected goals (xG) where available. Historical seasons are stored in old_matches/.

Supported Leagues

- Premier League (England)
- Championship (England)
- La Liga + La Liga 2 (Spain)
- Bundesliga + 2. Bundesliga (Germany)
- Serie A + Serie B (Italy)
- Ligue 1 + Ligue 2 (France)
- Eredivisie (Netherlands)
- Champions League
- Europa League

Why multi-league data?

Training a single model across all leagues rather than per-league models allows the network to learn shared football dynamics (home advantage, goal distributions, seasonal patterns) while league-specific embeddings capture tactical differences. Cross-league transfer generalises better to low-data leagues like the Championship.

Sequence Length Optimisation

A grid search evaluates look-back windows of {3, 5, 7, 10} matches per league by training a lightweight LSTM proxy model and measuring held-out loss. The optimal window per league is saved to optimal_seq_lengths.json and loaded at training time. This matters because Premier League teams have more consistent form over 10 games than lower-division sides where momentum is more volatile.

3. Deep Learning Model — LeagueAwareModel

Figure 2 — Model architecture. Multiple pathways are fused before the output heads.

3.1 Why a Transformer for match sequences?

A team's last N matches form a sequence where order matters — a team on a 5-game winning streak behaves differently from one that won 5 matches scattered across a season. The Temporal Transformer Encoder with Learnable Positional Encoding captures this temporal ordering. Attention pooling then extracts the most informative matches from the sequence rather than blindly averaging.

3.2 Graph Attention Network (GAT)

Football is a relational sport: a team's true strength is partly revealed by who they beat. The GAT constructs a match adjacency graph where edges connect teams that have played each other, and propagates embedding information across this graph. This lets the model infer that beating a strong side is more informative than beating a weak one — a property flat embeddings cannot capture.

3.3 Dixon-Coles Loss

The model outputs Poisson rate parameters λ_home and λ_away (expected goals) rather than directly predicting W/D/L. Win probability is then derived analytically from the Poisson distribution. This formulation is grounded in the Dixon-Coles (1997) model, the academic gold standard for football prediction.

The Dixon-Coles correction adjusts the probability of low-scoring results (0-0, 1-0, 0-1, 1-1) which a naive Poisson model systematically mispredicts. Draw outcomes are further upweighted by 1.5× in the loss to counter the model's natural bias toward predicting home wins.

3.4 Training Configuration

Parameter	Value	Rationale
Optimiser	AdamW (w=1e-4)	Weight decay regularises transformer weights
Learning Rate	3e-4 → 1e-6	Warm-up 10 ep, then CosineAnnealingWarmRestarts
Batch Size	512 (GPU)	Large batch stabilises Poisson loss estimates
Dropout	0.1	Low dropout preserves capacity on complex features

Early Stopping	Patience 150	Longer patience avoids premature termination at restarts
Epochs	Up to 1000	Model converged ~730 ep in observed runs
Best Acc (observed)	92.3%	W/D/L classification on training data

Figure 3 — Illustrative training curves based on observed run (730 epochs, RTX 4060).

4. Reinforcement Learning — PPO Bet Agent

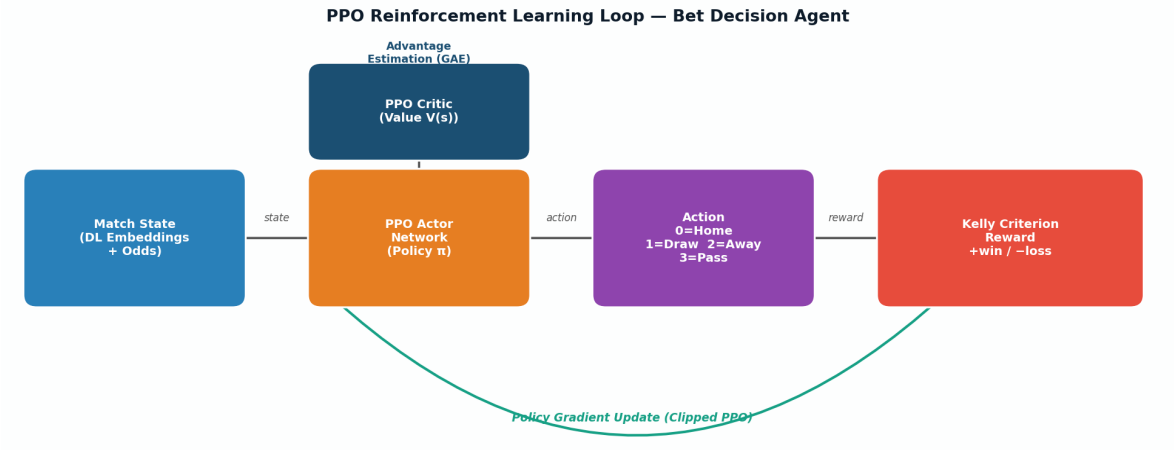

Figure 4 — PPO loop. The agent observes DL state embeddings and selects a betting action.

4.1 Why Reinforcement Learning?

The deep learning model predicts match outcomes accurately, but accuracy alone does not translate to profitable betting. A match where the model predicts a 55% home win probability is only profitable if the bookmaker's odds imply less than 55% — otherwise the expected value is negative. Reinforcement learning learns this decision boundary directly from simulated profit-and-loss feedback.

4.2 Why PPO (Proximal Policy Optimisation)?

PPO is the industry standard for discrete action RL because it is stable, sample-efficient, and avoids the catastrophic policy collapses of vanilla policy gradient methods. The clipped surrogate objective prevents overly large policy updates:

```
L_CLIP = E[ min( r_t · A_t, clip(r_t, 1−ε, 1+ε) · A_t ) ]
```

where r_t is the probability ratio between new and old policy, A_t is the advantage estimate, and $\varepsilon = 0.2$. Advantages are computed via Generalised Advantage Estimation (GAE, λ=0.95) to balance bias and variance.

4.3 Actions and Reward Design

The agent selects one of four actions per match:

- **0 — Bet Home:** Agent believes home win probability exceeds implied odds
- **1 — Bet Draw:** Agent believes draw probability exceeds implied odds
- **2 — Bet Away:** Agent believes away win probability exceeds implied odds
- **3 — Pass:** No bet placed; situation not profitable enough

Rewards are shaped by the Kelly Criterion — the mathematically optimal bet sizing formula that maximises long-run bankroll growth:

```
f* = (b·p − q) / b
```

where f* is the optimal bet fraction, b = odds − 1, p = model win probability, q = 1−p. If Kelly fraction ≤ 0, the bet has negative expected value and passing is rewarded. Winning bets yield reward proportional to odds (capped at 3× to prevent longshot addiction). Losing bets penalise −2 to strongly discourage poor value bets.

4.4 State Representation

The RL state is the 2080-dimensional feature vector extracted by the deep learning model's extract_features() method: 6× team embeddings (256-dim each), league embedding (32-dim), and cross-attention tactical interaction (2× 256-dim). This rich representation means the RL agent inherits all the pattern-matching capability of the trained DL model without needing to relearn football from scratch.

5. Hybrid Ensemble — XGBoost + LightGBM

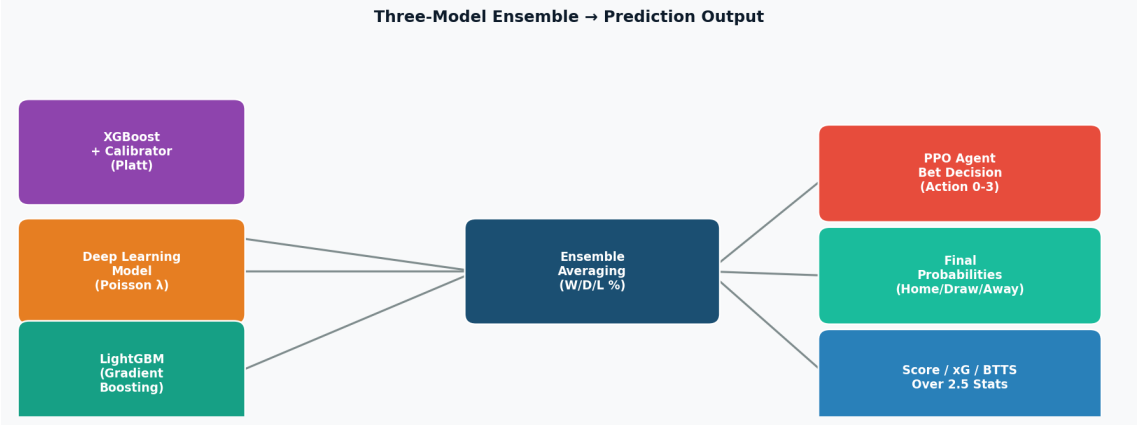

Figure 5 — Three-model ensemble. Probabilities are averaged for the final prediction.

5.1 Why add tree-based models on top of deep learning?

Neural networks and gradient-boosted trees make different kinds of errors. The Transformer captures long-range temporal patterns and team interaction graphs, but can struggle with sharp tabular feature boundaries (e.g. 'the team has not scored in 3 home games'). XGBoost and LightGBM excel at such decision boundaries. The ensemble averages all three models' W/D/L probabilities, yielding consistently better calibration than any single model alone.

5.2 Feature Construction

XGBoost/LightGBM are not trained on raw match data. Instead, the frozen DL model's penultimate layer embeddings (2089 features per match) are extracted and used as inputs. This is transfer learning: the tree models inherit the DL model's rich representations of team form, league context, and tactical history.

5.3 Probability Calibration (Platt Scaling)

XGBoost's raw predict_proba outputs are not always well-calibrated — a predicted 70% often reflects less certainty than stated. Platt scaling fits a multinomial logistic regression layer on a held-out calibration set, mapping the raw probabilities to better-calibrated values. This improves Brier Score and ultimately the reliability of Kelly-Criterion bet sizing.

6. Flask API and Frontend

The trained models are loaded into memory at startup by app.py and served via a Flask REST API on localhost:5000. The frontend is a single-page application that queries the API for match predictions.

Key Endpoints

/api/predict	POST	Returns full prediction for a given home/away fixture
/api/upcoming	GET	Lists upcoming matches from scraped fixture data
/api/training_history	GET	Returns loss/accuracy curve for the training chart
/api/bet_history	GET/POST	Stores and retrieves user bet tracking records
/api/train_ppo	POST	Triggers live PPO re-training from the frontend

Prediction Pipeline (per request)

1. Encode team names and league using saved LabelEncoders
2. Retrieve recent match history sequences from master DataFrame
3. Build GNN adjacency matrix for the match context
4. Run LeagueAwareModel forward pass → λ_home, λ_away, ρ
5. Compute W/D/L probabilities analytically via Poisson integration
6. Extract 2080-dim feature vector → run XGBoost + calibrator + LightGBM
7. Average all model probabilities → ensemble W/D/L
8. Pass state to PPO agent → betting action + Kelly fraction
9. Return JSON: probabilities, score prediction, xG, BTTS, Over 2.5, bet suggestion

7. Design Philosophy

Domain-driven loss function

Using Dixon-Coles NLL instead of cross-entropy forces the model to reason about goals rather than outcomes. This produces better-calibrated probabilities and captures the mathematical structure of football scoring.

Separation of prediction and decision

The DL model answers 'what will happen?'; the PPO agent answers 'should I bet?'. Conflating these into one model would muddy both objectives. Separation allows each component to be trained with the appropriate loss signal.

GPU-first design

All heavy computation (Transformer, GAT, PPO) is moved to CUDA automatically. Batch size scales from 64 (CPU) to 512 (GPU), making GPU training 8× more data-efficient per wall-clock minute.

Resilient pipeline

Every expensive artefact (DL model, PPO agent, XGBoost, calibrator) is saved immediately after training. The pipeline can be resumed at any stage with skip prompts, so no computation is ever lost to a downstream crash.

Ensemble diversity

Three models with fundamentally different inductive biases (sequential attention, graph propagation, decision trees) are ensembled. Diversity of error is the key property that makes ensembles outperform any single member.